

AI-Powered Software Product Development

From Vibe Coding to high autonomy — a survey of methodologies & frameworks 2024–2026



Presented to the R&D · Product · QC teams

Data sources: DORA 2025, Stack Overflow 2025, GitHub Octoverse 2025, METR RCT, independent benchmarks — citation links on each slide

Agenda — 8 sections, ~75 minutes



1. The problem with pure vibe coding



5. Mapping Waterfall / Prototype / Scrum / XP



2. The L1–L5 autonomy spectrum + 4 engineering layers



6. Consolidated comparison & industry data



3. Spec-driven methodologies



7. Selection matrix & counterarguments



4. Agentic tooling



8. Roadmap + guardrails for VN fintech



Pure vibe coding is not enough for fintech production

- Vibe coding = generating software from prompts, accepting the output without carefully reading the code (term coined by A. Karpathy, Feb 2025).
- ~72% of developers say vibe coding is NOT part of professional work; another 5% flatly refuse to use it.
- Consequences: fragile codebase, context lost between sessions, no audit trail — below the bar for regulated fintech.
- Good fit: prototypes, PoCs, internal tools that never touch real data.

84%

of devs use or plan to use
AI tools

29%

trust the accuracy of AI

66%

frustration #1: code
“almost right, but not
quite”

46%

actively DISTRUST AI (>
the 33% who trust)

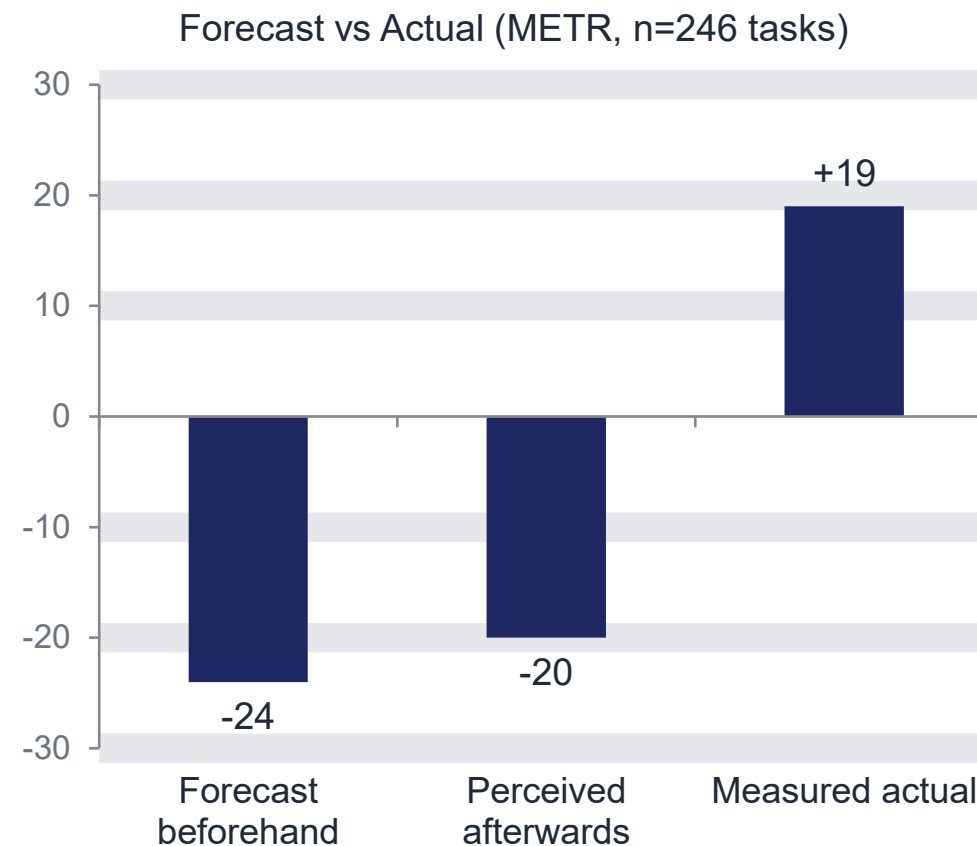
“AI solutions that are almost right, but not quite”

— Developers’ #1 frustration — Stack Overflow Developer Survey 2025 (49,000+ responses)



The productivity paradox: METR's RCT (Jul 2025)

- 16 experienced developers (avg 5 yrs, ~1,500 commits on their own repos), 246 real tasks, Cursor Pro + Claude 3.5/3.7.
- Forecast beforehand: AI makes them 24% faster. Perceived afterwards: 20% faster.
- **Actual measurement: 19% SLOWER when allowed to use AI (95% CI: +2% to +39%).**
- Causes: models weak on complex cases/familiar codebases, prompt + review time, distraction.
- *Limits: applies only to experts on familiar repos — does NOT negate gains on boilerplate, greenfield, unfamiliar codebases.*





But adoption is irreversible

80%

of NEW devs on GitHub use Copilot in their very first week

+178%

YoY growth in public repos using LLM SDKs (1.1M repos; ~694K created in 12 months)

1M+

PRs created by the Copilot coding agent in just 5 months (May–Sep 2025)

90%

of software professionals use AI at work (DORA 2025, +14% YoY)

- TypeScript overtook Python & JavaScript as the #1 language (Aug 2025) — type safety becomes a guardrail for AI-generated code.
- 2025 academic research: 94% of LLM code compile errors are type-check failures → pick a stack with strong type checking.
- **The right question is not “should we use AI” but “how do we use it safely and under control”.**



The autonomy spectrum: 5 levels — analogous to SAE for self-driving cars

Risk level & supervision requirements increase →

L1 — Autocomplete

Inline suggestions. Classic Copilot.

L2 — Chat-assisted

Q&A, generating code blocks. The human manages context.

L3 — Agent + supervision

Agent edits multiple files, runs commands; human supervises. Cursor agent, Claude Code, Cascade.

L4 — Autonomous / async

Agent runs long tasks in the background; humans audit. Devin, Copilot agent, Background Agent.

L5 — Multi-agent fleet

“Dark factory” — AI does its own QA. Hypothetical for production.

“L3 is the current production ceiling” — asdlc.io. Higher ≠ better (Swarmia).



Risk rises with the autonomy level



Paradox of Supervision

The longer an agent runs independently, the more human supervision skills atrophy — right when they are needed most.



Silent Drift

Code compiles, tests pass, yet it drifts away from the original intent — hard to catch with ordinary review.



Token multiplication

Multi-agent burns ~15× the tokens of a single chat (Anthropic data); cost grows non-linearly with level.



Expanded attack surface

Agents can run commands/access tools → prompt injection becomes a real operational risk.

“higher isn’t always better”

— Swarmia, on levels of AI coding autonomy

Four engineering layers — from a command to a system

1. PROMPT

Design clear prompts so the AI produces the desired result in ONE shot.

When to use?

- Writing/editing content, summarizing
- Q&A, information extraction
- Simple one-step AI tasks

How it flows



In this deck: vibe coding, L1–L2.

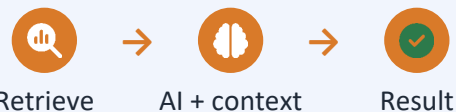
2. CONTEXT

Give the AI the right information & context BEFORE it answers.

When to use?

- RAG (retrieval + generation)
- AI copilots / knowledge assistants
- Needs memory, reference documents

How it flows



In this deck: CLAUDE.md, spec, memory bank.

3. HARNESS

An orchestration layer AROUND the AI so the system runs accurately, safely, reliably.

When to use?

- Production AI applications
- Tool-using agents
- Workflows needing high reliability

How it flows



In this deck: hooks, CI gates, fintech guardrails.

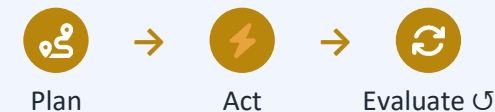
4. LOOP

A system where the AI plans → acts → self-evaluates → retries until done.

When to use?

- Coding agents
- Research agents
- Automating multi-step tasks

How it flows



In this deck: agent mode, Devin, L3–L4.

Evolution: Prompt → AI can answer · Context → AI understands the problem · Harness → AI operates safely · Loop → AI finishes on its own.

Mapping: which layers does each method / tool cover?

Method / Tool	Prompt	Context	Harness	Loop	Practical focus
Vibe coding	●	—	—	—	Prompts only — no durable context, no guardrails
Context engineering (CLAUDE.md, rules, memory)	●	●	—	—	Loads the right context before the AI runs
OpenSpec	●	●	●	—	Spec = durable context; the review gate stays human
GitHub Spec Kit	●	●	●	—	constitution + /clarify, /analyze = process-level harness
Amazon Kiro	●	●	●	●	EARS + Agent Hooks (file events) + tasks run in waves
BMAD-METHOD	●	●	●	●	Agent roles + QA gate; ceremony acts as the harness
Agent OS · Task Master	—	●	●	●	Standards layer + task decomposition for agents
Claude Code (hooks, subagents, MCP)	●	●	●	●	Hooks = harness deterministic; Plan Mode → loop
Cursor / Copilot agent mode	●	●	●	●	Agent self-loops fix–run–test; harness relies on CI
Devin · Background Agent (L4)	●	●	●	●	Closed loop in a sandbox; harness = CI + human PR review
LangGraph · CrewAI · AutoGen	—	●	●	●	Frameworks for BUILDING your own harness + loop for agents

● full coverage
 ● partial coverage
 — no coverage

Reading the table: no tool covers all 4 layers. Fintech production must COMBINE: methodology (Context) + hooks/CI (Harness) + agent mode (Loop).



Two solution groups — you need both

GROUP 1 — Spec-driven methodologies (define the PROCESS)



BMAD-METHOD

multi-agent simulation of an agile team



GitHub Spec Kit

/specify → /plan → /tasks → /implement



OpenSpec

proposal-based, brownfield-first



Amazon Kiro

requirements (EARS) + design + tasks



Agent OS · Task Master

standards layer · PRD → tasks

GROUP 2 — Tooling / orchestration (the VEHICLE)



Claude Code

terminal-first: subagents, hooks, MCP, Skills



Cursor · Windsurf

IDE agentic; Background Agent (L4)



GitHub Copilot

agent mode + coding agent; compliance mature



Devin · OpenHands · Jules

autonomous cloud agents (L4)

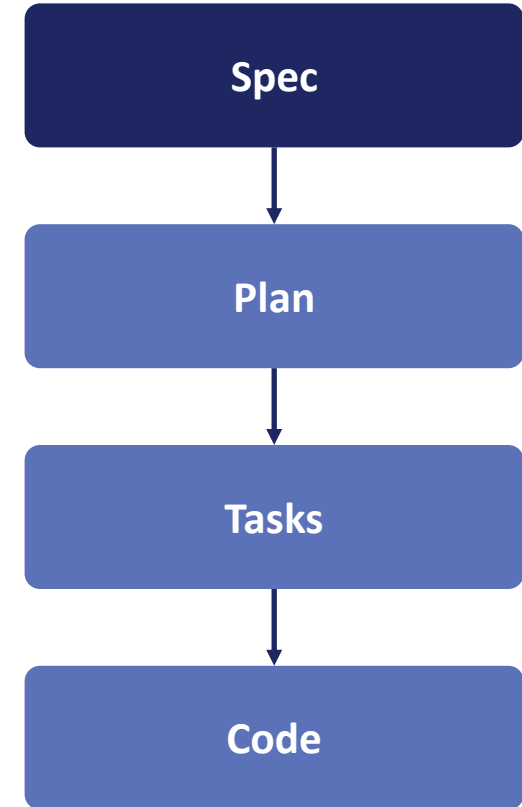


LangGraph · CrewAI · AutoGen

multi-agent orchestration frameworks

Spec-Driven Development — “spec is the new source code”

- Spec = a version-controlled artifact, NOT a document thrown away once the code ships.
- Replace one-shot prompts with multi-step refinement: each phase produces an artifact feeding the next (spec → plan → tasks → code).
- Sean Grove (OpenAI), “The New Code” talk (AI Engineer 2025): the value lives in the structured spec — code is just a “lossy projection” of intent.
- GitHub formalized this direction with Spec Kit (Sep 2025): “spec-driven development with AI” as an open-source toolkit.
- The direct opposite of vibe coding: intent is recorded, reviewable, auditable.



each artifact goes into git → audit trail



Context Engineering — the #1 skill of the AI coding era

- Definition (Tobi Lütke, standardized by Simon Willison, Jun 2025): the art of providing ENOUGH context to make the task feasible for the LLM.
- Developers shift from writing every line of code → managing the context the model gets access to.
- Willison likens an LLM to a junior developer who has read every textbook yet has 0 experience with your codebase, and forgets everything older than ~1 hour.
- Practical consequence: CLAUDE.md / rules / steering docs / memory bank / spec — all are context engineering tools.
- **Replaces “prompt engineering”: the problem is not a clever prompt, it is what the model knows at execution time.**

“the art of providing all the context for the task to be plausibly solvable”

— Simon Willison, “Context engineering”, simonwillison.net (Jun 27, 2025)



BMAD-METHOD — simulating an entire agile team with agents

github.com/bmad-code-org/BMAD-METHOD

- v6: 19–21 specialized agents (Analyst, PM, Architect, Scrum Master, PO, Dev, QA/Test Architect, UX...) + 50+ workflows.
- Two phases: Planning (brief → PRD → architecture) then story-driven Implementation; includes sprint-planning & retrospective.
- “Scale-adaptive intelligence”: planning depth adjusts to complexity; Web Bundles push planning onto a subscription to save tokens.
- Pros: highest spec/architecture quality of the group; full audit trail; suits legacy modernization.
- Cons: ~2-month learning curve; heavy artifacts hard to review; high token cost (large users: ~230M tokens/week).

~49K

GitHub stars (6/2026)

6 days

BMAD Full / 1 feature
(independent
benchmark)

~\$200

cost / cycle (BMAD Full)

3.65/5

DX score (BMAD Full;
Quick: 3.74)

Use case: large/complex systems, team ≥5,
architectural consistency required. NOT for bug
fixes / small features.

Workflow:

Brief



PRD



Architecture



Epics/Stories



Dev + QA



Retro



GitHub Spec Kit — governance layer, agent-agnostic

github.com/github/spec-kit

- Standard flow: /speckit.specify → /plan → /tasks → /implement; extra gates: /constitution, /clarify, /checklist, /analyze.
- constitution.md = the project's non-negotiable principles (testing standards, conventions) — every agent validates specs against it.
- 30+ agent integrations (Copilot, Claude, Cursor, Gemini, Codex, Windsurf...) → no vendor lock-in.
- Pros: traceability, clear review gates, standardization for teams.
- Cons: heavy, rigid phase gates, “sea of markdown”; benchmark: weak at implementation — code does not faithfully map to spec (DX 2.77/5, lowest of the 3 tools tested).

111K

GitHub stars (Jun 2026)
— fastest-growing of
the group

~30 min

setup (requires
Python/uv)

~\$75

/ feature cycle
(benchmark: ~1.5 days)

2.77/5

DX score — weak at
the implement step

Use case: greenfield / 0→1, teams needing governance & no lock-in. Criticized as “reinvented waterfall” when applied mechanically.

Workflow:

/constitution



/specify



/plan



/tasks



/implement



OpenSpec — the lightest, brownfield-first

github.com/Fission-AI/OpenSpec

- Proposal-based: /opsx:propose → specs/design/tasks → /opsx:apply → /opsx:archive.
- Two folders: openspec/specs/ (current source of truth) + openspec/changes/ (one folder per change) — “version control for intent”.
- Philosophy: “fluid not rigid, iterative not waterfall”; installs via npm, no Python/API key required.
- Pros: ~5-minute setup; lean specs (~250 lines vs ~800 for Spec Kit); easy to review; suits legacy codebases.
- Cons: does not force the AI to follow the spec; less governance than Spec Kit; file-based, prone to merge conflicts.

~54K

GitHub stars (Jun 2026;
repo created Aug 2025)

~5 min

setup via npm

4.00/5

DX score — HIGHEST of
the 3 tools tested

~\$95

/ feature cycle (~1.5
days)

Use case: legacy/brownfield, small–medium features, teams wanting minimal structure. Recommended as the pilot starting point.

Workflow:

/opsx:propose



specs + design + tasks



/opsx:apply



/opsx:archive



Amazon Kiro — a spec-driven IDE with EARS notation

kiro.dev

- Three files per feature: requirements.md (EARS) → design.md (architecture, mermaid, data model) → tasks.md (atomic checklist).
- Agent Hooks: IDE-level triggers on file events (save/create/edit) — unlike GitHub Actions (repo events).
- Runs tasks in parallel by dependency “waves”; \$19/month; native AWS integration.
- Pros: EARS is testable, supports property-based testing; the spec stack in VCS = an audit trail of intent.
- Cons: front-loaded overhead per feature; criticized for “over-specification”; poor fit for small edits/bug fixes.

EARS notation example:

```
WHEN the user enters a wrong OTP  
3 times  
THE SYSTEM SHALL lock the  
transaction  
and send an alert within 5  
seconds  
→ testable, auditable clauses
```

Use case

AWS/cloud-native teams; medium–large features needing design docs + audit trail. Property-based testing catches edge cases unit tests miss.

Workflow:

requirements.md (EARS)



design.md



tasks.md



execute in waves



The supporting layer: standards, tasks, memory



Agent OS

Builder Methods (Brian Casel) · ~4.9K stars · free.

Injects coding standards + shapes specs; v3 (2026) defers spec-writing to Claude Code/Cursor Plan Mode.

Commands: `/shape-spec`, `/discover-standards`, `/inject-standards`; `orchestrate-tasks` delegates to subagents.

Good for: keeping code standards consistent across projects; unearthing tribal knowledge in legacy.



Task Master

eyaltoledano/claude-task-master · ~27K stars.

Parses a PRD → generates & tracks the task list; used with Claude Code/Cursor.

Now the de-facto standard for PRD-driven workflows in the agentic ecosystem.

Good for: managing the agent task backlog when a PRD already exists.



Cline Memory Bank

A markdown file structure persisting durable project context for the Cline agent (open-source, BYOM).

Tackles the “AI forgets after every session” problem head-on.

A good reference pattern for building your own internal memory convention, even without Cline.

Common trait: they do NOT replace the coding assistant — they add a standards / task / memory layer on top.



Claude Code — 6 primitives for agentic workflows

anthropic.com/claude-code · docs: docs.claude.com



CLAUDE.md

the repo's constitution, loaded every session
(always costs tokens)



Skills

SKILL.md loads on demand (~100 tokens
when idle)



Subagents

context isolation, parallel runs, separate tool
allowlists



Slash commands

package repeated workflows into commands



Hooks

deterministic, runs OUTSIDE the model's
context, cannot be overridden



MCP

open standard for connecting tools/data
(Anthropic open-sourced, Nov 2024)

“CLAUDE.md and skills are a REQUEST — only hooks are a GUARANTEE”

— Operating principle → for fintech: security/compliance checks MUST be hooks (PreToolUse/PostToolUse), not prompt lines. Also has Plan Mode & background agents (L4).



Flagship agentic tooling — 2026 standings



Cursor

VS Code-fork IDE · SOC 2 certified → viable for enterprise.
Background Agent (L4) reaches 65.7% on SWE-bench Verified.
Rules via `.cursor/rules/*.mdc`; credit-based billing hard to predict (~\$2,000/month / 50 devs).



Windsurf (Cascade) → Cognition

Cognition acquired Windsurf (Jul 2025), merging it with the Devin engine.
SWE-1.5 model: up to ~950 tok/s (via Cerebras).
Cascade Agent: multi-file edit, fast context retrieval.



GitHub Copilot

The most mature compliance + IP indemnity → a safe enterprise foundation.
Agent mode + autonomous coding agent (assign an issue → receive a PR).
~\$1,950/month for a 50-dev team (Enterprise).



Devin (Cognition)

Autonomous agent (L4) in a VM sandbox; takes tickets via Slack/Linear/MCP.
Devin 2.0: \$500 → \$20/month; 45.8% SWE-bench (independent tests: 15–30%).
Inside Cognition: 154 → 659 Devin PRs merged/week (2025 → early 2026).

Warning: Google Antigravity — security flaws within the first 24h, no compliance cert yet → do NOT use in production. antigravity.google



Benchmarks & the multi-agent debate (Jun 2025)

- SWE-bench Verified (500 human-reviewed tasks) is the reference — but saturated on isolated tasks.
- Same base model, different scaffolds differ by 15+ points: Cursor Background 65.7% · Factory Droid 58.1% · Devin 2.0 45.8%.
- SWE-EVO shows: ~80% of real effort is maintaining/evolving legacy — benchmarks do not measure this.

Cognition: “Don’t Build Multi-Agents”

Parallel subagents share context poorly → conflicting decisions → compounding errors. Recommendation: single-threaded agent + full context.

Anthropic: multi-agent research system

Lead agent (Opus 4) + subagents (Sonnet 4) beat single-agent by 90.2% on internal research evals — but at ~15× tokens; token usage explains 80% of performance variance.

Pragmatic conclusion: multi-agent suits READ tasks (research); risky for WRITE tasks (coding) → fintech should prefer single-thread + gates.



Mapping #1: Waterfall ↔ Spec-Driven Development

Traditional Waterfall

- Spec is LOCKED after the phase gate; changing it = a costly change request.
- Build cycles measured in MONTHS.
- Feedback from reality arrives too late.

SDD (Spec Kit / Kiro)

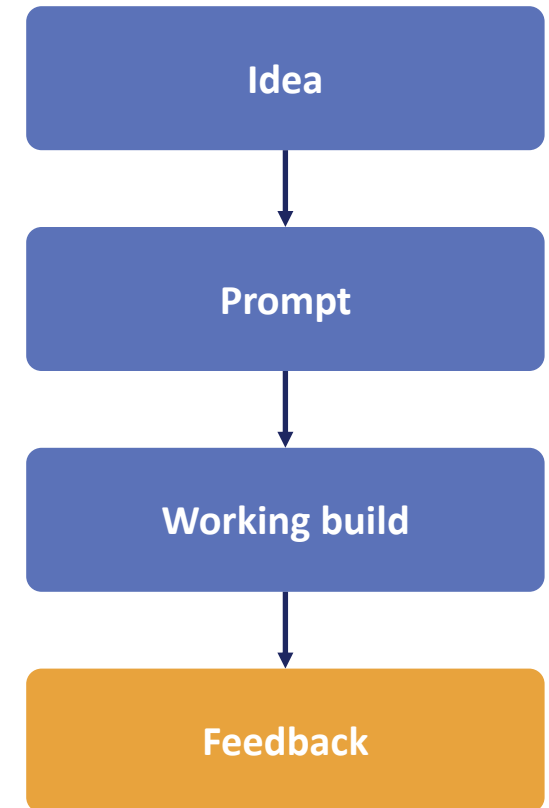
- Spec is a **LIVING ARTIFACT** — continuously updated from implementation feedback (“spec mutability”).
- The spec → generate → evaluate → regenerate cycle takes MINUTES/HOURS.
- Think of it as a “mini-waterfall per feature” — upfront discipline, agile speed.

THE TRAP: without tooling/review discipline, SDD generates thousands of AI-written spec lines nobody reads → “big design upfront” reborn. Critics call SDD a “Waterfall renaissance” (Scott Logic, Marmelab) — the rebuttal lies in spec mutability + short cycles.



Mapping #2: Prototyping / RAD ↔ Vibe coding

- Vibe coding = the direct descendant of Rapid Prototyping: building incrementally in natural language, “no mockups needed”.
- Marmelab calls this “Natural Language Development” — coding agents “supercharge Agile” rather than replace it.
- The idea → working build loop shrinks from weeks to minutes: an OpenSpec benchmark produced a complete website in ~7 minutes.
- Right place: exploring ideas, stakeholder demos, PoCs, hackathons.
- **Wrong place: letting a prototype go “straight to” production — fragile codebase, no audit trail. Prototypes must be RE-BUILT with a spec before entering production.**



loop: minutes, not weeks



Mapping #3: Scrum / Agile ↔ BMAD

Traditional Scrum	BMAD (AI-driven)
Scrum roles (PO, SM, Dev, QA...)	Specialized agents (PM, Architect, SM, Dev, QA/Test Architect, UX...)
Product backlog · epics · user stories	Epics → stories — each story runs in its own fresh chat/context
Sprint	Agent task cycle (sprint-planning workflow)
Sprint review / retrospective	Retrospective workflow built into v6
Definition of Done	Story acceptance criteria + QA agent gate

BMAD is the most direct mapping of Agile ceremonies into the AI world — so it also inherits the ceremony cost: benchmark shows 6 days/feature on BMAD Full.



Mapping #4: XP / Kanban ↔ Agentic workflow



Pair programming (XP)

AI pair programmer: Copilot/Cursor chat — human drives, AI assists; agent mode flips the roles: AI drives, human reviews.



TDD (XP)

Test suite = “superpowers” for agents to self-validate & iterate; red/green TDD is 1 of the 3 patterns Willison uses daily. A spec with expected outcomes = TDD at the requirements level.



Kanban board

Dashboards visualizing the agent task pipeline: Spec Kitty, OpenSpec UI, Mission Control (Cursor).



Small batches / WIP limit

DORA 2025: “working in small batches” is 1 of the 7 capabilities that decide AI success — its value is unchanged with agents.

Takeaway: *test suites let agents self-verify and iterate — XP’s value is unchanged in the agentic era (Willison, DORA 2025).*



What CHANGES — what STAYS CONSTANT

CHANGES

- Role: writer → reviewer / orchestrator (Octoverse: “strategic orchestrators”).
- Spec / documentation become first-class artifacts, living in git.
- Feedback loop: months → hours / minutes.
- The priciest skills: context engineering + verification.

STAYS CONSTANT

- Still need planning, clear architecture, review/QA, tests.
- Humans are the “ultimate arbiter of quality” — 75% of devs still ask a person when they distrust AI (SO 2025).
- Process quality determines outcomes — not the tool.
- Trust but verify: ~30% “little or no trust” is a healthy level (DORA).

“successful AI adoption is a systems problem, not a tools problem”

— DORA 2025 — State of AI-assisted Software Development



Consolidated comparison table

Criterion	Vibe coding	OpenSpec	Spec Kit	Kiro	BMAD	CC / Cursor	Devin
Structure level	Very low	Medium	High	High	Very high	Low–Med (setup-dependent)	Med
Autonomy level	L2	L3	L3	L3–L4	L3	L3–L4	L4
Upfront investment	~0	~5 min	~30 min	Med (IDE)	High (~2 months)	Med	Low
Reference cost	Low	~\$95/cycle	~\$75/cycle	\$19/month	~\$200/cycle	per token	\$20–200/mo
Long-term context	Very poor	Good (specs/)	Good (artifacts)	Good (VCS)	Very good	Med (needs memory)	Med
Enterprise / audit	No	Yes (add gates)	Yes	Yes (AWS)	Yes (ledger)	Copilot/Cursor: yes	Caution
DX score (benchmark)	—	4.00 ★	2.77	—	3.65 / 3.74	—	—

DX score: independent 1-feature benchmark, Cursor + Claude Sonnet 4.5 (Ran the Builder) — a single observation, not a constant. Stars/prices change fast: re-check before deciding.



Industry data #1: DORA 2025 — AI is a “mirror and multiplier”

90%

of professionals use AI at work
(+14% YoY, ~5,000 respondents)

59%

report a positive impact on code
quality



throughput UP (reversing 2024)
but instability UP as well

~30%

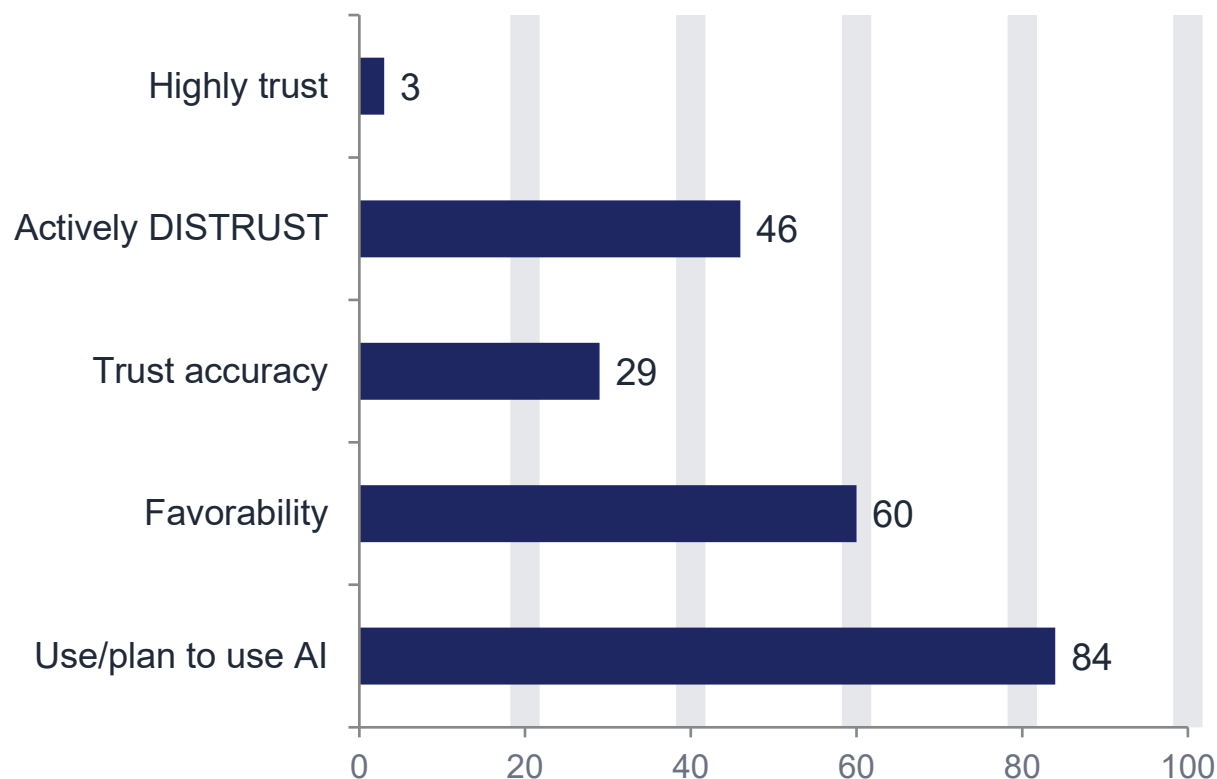
“little or no trust” — a level
DORA considers healthy

- **AI amplifies the team: strong teams get stronger, weak teams get exposed — “mirror and multiplier”.**
- Change volume rises → instability rises without automated tests, mature VCS, fast feedback.
- 7 deciding capabilities: a clear AI strategy, healthy data ecosystem, platform engineering, user-centric focus, small batches...
- **Implication for us: invest in process + platform BEFORE raising the autonomy level.**



Industry data #2: trust falls even as adoption rises

Stack Overflow 2025 (49,000+ respondents)



- Trust in accuracy: 40% → 29% within a year; favorability 72% → 60%.
- Experienced devs are the most cautious: only 2.6% highly trust; 20% highly distrust.
- 66% cite the “almost right, but not quite” frustration; 45% lose time debugging AI code.
- AI agents not yet mainstream: 52% do not use them or use only simple tools.
- **Combined with METR (–19% for experts): keep expectations realistic, measure instead of trusting gut feel.**



Case study: 3 SDD tools, same feature, same codebase

Test conditions: Cursor + Claude Sonnet 4.5, same feature, same codebase — [Ran the Builder \(Itzhak Eretz Kdosha\)](#)

Tool	Time	Cost	DX score	Key remarks
OpenSpec	~1.5 days	~\$95	4.00 ★	Overall winner: lean, easy to review, free iteration
BMAD Quick	~2 days	~\$85	3.74	Good balance of structure and speed
BMAD Full	6 days (2 plan + 4 implement)	~\$200	3.65	Deepest spec/architecture; heavy on ceremony
Spec Kit	~1.5 days	~\$75	2.77	Cheapest, but code does not faithfully map to spec intent

- *Mandatory caveat: this is a single test on one feature — “a single-test observation, not a general multiplier”. Do not over-generalize.*
- **But the pattern matches other independent reviews: light + reviewable wins on ordinary features; heavy only pays off on large modules.**



Critical risks — fintech is a high-value target



Prompt injection: Willison: the “lethal trifecta” (private data + untrusted content + external comm) — a problem with NO complete solution yet. The more agent permissions, the wider the surface.



Shadow AI: 38% of employees admit sharing confidential data with unauthorized AI tools → need a policy + an approved AI channel.



Technical debt & instability: DORA: rising throughput drags instability up without automated tests + mature VCS + fast feedback.



Spec drift: AI-generated specs without human review → code drifts from intent; the Spec Kit benchmark clearly illustrates the spec-code gap.



Skill atrophy: Paradox of Supervision: long-term reliance on agents erodes the very skills needed to supervise them.

Bottom-line rule: PII / customer data / credentials NEVER go into an uncontrolled LLM.



Selection matrix — firm recommendations

Project type	Team	Recommendation	Why
Internal prototype / PoC (no real data)	1–3	Vibe coding + Cursor / Claude Code (L2–L3)	Speed; low risk since it never reaches production
Small–medium features on legacy / brownfield	2–6	OpenSpec + review PR	Brownfield-first, 5-minute setup, top DX
New production feature (greenfield)	3–10	Spec Kit + constitution + /clarify + /analyze	Governance, traceability, no lock-in
Large module / system-of-systems, intent audit required	≥5–8	BMAD (or Kiro if AWS) + PR gates	Architectural consistency, compliance ledger
Repetitive tasks: tested bug fixes, migrations, dep updates	any team	Copilot agent / Cursor Background (L4) + CI gate	Async, scales; tests are the guardrail
Org-wide platform (regulated)	org	Copilot Enterprise / Cursor Teams (SOC 2)	IP indemnity, audit trail, security posture

Counterpoint: when NOT to use

-  **DO NOT use BMAD for small projects**
The orchestration cost (6 days, ~\$200/cycle, ~2 months to learn) is not justified — textbook over-engineering.
-  **DO NOT run multi-agent fleets for coding**
~15× tokens, coordination failures, silent drift — single-thread + gates is safer for write tasks.
-  **DO NOT treat SWE-bench as a production yardstick**
Saturated on isolated tasks; does not measure maintenance/evolution (~80% of real effort).
-  **DO NOT let tools without compliance certs touch production**
Antigravity: security flaws found within the first 24h; no cert yet.
-  **DO NOT trust AI-generated specs without human review**
Spec drift turns SDD into fake “big design upfront” — all audit value lost.



Adoption roadmap — 4 phases, with promotion & rollback criteria

P1 · months 0–3

AI-assisted (L1–L2)

Copilot/Cursor for the whole team; autocomplete + chat only; data policy; 100% human review.

Advance when: tools familiar, policy in place, DORA baseline measured.

P2 · months 3–6

Spec-driven (L3)

OpenSpec (brownfield) / Spec Kit (greenfield); constitution encodes SBV standards; specs go into git.

Advance when: spec review has become a habit; instability is not rising.

P3 · months 6–12

Agentic supervised (L3–L4)

Agent mode + human-in-the-loop; MANDATORY hooks: security scan, lint, tests, secret detection.

Advance when: hooks/CI stable; full audit trail; good MTTR.

P4 · months 12+

Autonomous with gates (L4)

Background agents for well-defined tasks with tests; VM sandbox; every PR passes human + security + compliance review.

ROLL BACK a phase if: change fail rate or security incidents rise.

DORA principle: start small → measure → expand deliberately. Every phase has promotion AND rollback benchmarks.



Guardrails for Vietnamese fintech



Legal framework

- Circular 64/2024/TT-NHNN (Open API) · Circular 45/2025/TT-NHNN (biometric identification).
- Decree 94/2025/NĐ-CP: fintech sandbox (effective Jul 1, 2025).
- AI Law No. 134/2025/QH15: effective Mar 1, 2026, 4-tier risk-based, penalties as % of revenue.



Audit trail

- Version-controlled specs + git history = evidence of intent for inspectors.
- A compliance ledger per PR (which human/agent authored, who approved, which gates passed).
- Full agent session logs (commands run, files changed).



Security

- SAST/DAST + secret scanning in CI — run as HOOKS, not prompts.
- Sandbox/VM for agents; allowlist-based permissions.
- Data localization; no training on customer data; block PII from reaching LLMs.



Review gates

- Mandatory human review for every agent-created PR.
- Willison's rule: never commit code you could not explain to somebody else.
- Measure change fail rate regularly to recalibrate the autonomy level.

Note: legal documents require formal review by legal/compliance before entering policy — this slide is context reference only.



Actions for the first 90 days

Weeks 1–2

Issue a data policy (PII/credentials never into LLMs) + an approved AI channel; pick the compliance platform (Copilot Enterprise / Cursor Teams).

Weeks 3–4

Measure the DORA baseline: lead time, deploy frequency, change fail rate, MTTR — without a baseline you cannot evaluate AI.

Weeks 5–8

Pilot OpenSpec on 1 real brownfield module (2–4 devs); context engineering training for the whole engineering team.

Weeks 9–12

Build compliance hooks (secret scan, SAST, test gate) into CI; pilot retrospective; decide on P2 expansion based on the data.

KPIs after 90 days: 1 module running OpenSpec end-to-end · 100% of agent PRs through hooks + human review · a complete DORA baseline.

Conclusion



There is no silver bullet — pick the framework by project type, not by hype.



AI is a mirror & multiplier: it amplifies your existing process, good and bad.



Humans are the final arbiter — the role shifts to reviewer/orchestrator.



Spec + verification + guardrails = the formula for regulated fintech.

“AI doesn’t eliminate engineering — it makes engineering a lot more important.”

The shared takeaway of DORA 2025 & the practitioner community



Q&A — Key references

Surveys & research

[DORA 2025 \(dora.dev\)](#)

[Stack Overflow Survey 2025](#)

[GitHub Octoverse 2025](#)

[METR RCT \(arXiv:2507.09089\)](#)

[metr.org](#)

[simonwillison.net](#)

[asdlc.io](#)

[Swarmia Blog](#)

Methodology (Group 1)

[github/spec-kit](#)

[GitHub Blog — SDD](#)

[Fission-AI/OpenSpec](#)

[BMAD-METHOD \(GitHub\)](#)

[kiro.dev](#)

[Agent OS](#)

[claude-task-master](#)

[Cline](#)

Tooling (Group 2)

[Claude Code](#)

[docs.claude.com](#)

[cursor.com](#)

[windsurf.com](#)

[devin.ai](#)

[GitHub Copilot](#)

[aider.chat](#)

[OpenAI Codex](#)

Analysis & benchmarks

[Ran the Builder — benchmark 3 SDD tools](#)

[rywalker.com/research](#)

[Cognition — Don't Build Multi-Agents](#)

[Anthropic — Multi-agent research system](#)

[swebench.com](#)

[Marmelab Blog](#)

[Scott Logic Blog](#)

[SBV \(sbv.gov.vn\)](#)

Others: [jules.google](#) · [antigravity.google](#) · [OpenHands](#) · [LangGraph](#) · [CrewAI](#) · [AutoGen](#)

All links are clickable in this file · Data current as of Jun 2026 — re-verify before official citation